Agile Software Development and DevOps Lab

Experiment 5

Name	: Aaditya Surve	**Roll No.**	: 2401119
Batch	: T22	**Experiment**	: 05

Aim: To automate application deployment using GitHub/Jenkins.

LO mapped: LO3

LO: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory

Continuous Integration (CI) and Continuous Deployment (CD) form a fundamental DevOps practice aimed at bridging the gap between development and operational activities.

- **Continuous Integration (CI):** A software development practice where developers regularly merge code changes into a central repository, followed by automated builds and automated tests. This helps catch bugs early, increases software quality, and reduces validation overhead.
- **Continuous Deployment (CD):** The automated process where verified changes in the codebase are automatically packaged, configured, and deployed directly to production or staging runtime environments without manual intervention.
- **Jenkins & Declarative Pipelines:** Jenkins is an open-source automation server enabling automated build, test, and deployment workflows. A Declarative Pipeline uses structured Groovy-based domain-specific language (DSL) to define stages, steps, and build environments as code (Jenkinsfile), making infrastructure version-controlled and reproducible.

Step 1: Jenkins Server Initialization and Access Setup

- Jenkins runs as a background Windows service requiring a compatible Java Development Kit (JDK 21) runtime environment to execute its core processes.
- Accessing the web console at http://localhost:8080 allows the setup wizard to be completed, during which the first administrator user is created to manage permissions, job configurations and server tasks.

Step 2: Baseline Pipeline Validation (Inline Scripting)

- A new project item named GitHub-Jenkins-Demo is created using the native Pipeline job type.
- Before linking to external version control, an inline Declarative Pipeline script is added directly to the job configuration to test the server's execution engine.
- The script utilizes the agent any directive along with three core lifecycle stages: Build, Test, and Deploy, each executing standard echo statements.
- Running Build #1 causes Jenkins to assign a local node executor in the workspace directory, process each stage sequentially, and display the console logs showing that the pipeline completed with SUCCESS.

Step 3: Source Code Management (SCM) & Pipeline-as-Code Implementation

- A central GitHub repository named jenkins-demo-app is established under user profile to maintain the application codebase and its automation configuration.
- An index.html file is added as the frontend application artifact containing an initial header message.
- A version-controlled Jenkinsfile is defined in the root directory, following Declarative Pipeline syntax.
- In the Deploy stage of the Jenkinsfile, automated batch instructions (bat) are scripted to verify the destination directory C:\Deployments\DemoApp and copy index.html from the workspace to the target folder.

Step 4: Connecting Jenkins to GitHub and Initial Deployment

- The Jenkins project configuration is transitioned from an inline script to Pipeline script from SCM, specifying Git as the source provider.
- Jenkins is supplied with the repository HTTPS URL, the branch specifier is configured to */main, and the script path is set to target the repository's Jenkinsfile.
- Executing Build #2 prompts Jenkins to clone the remote repository, check out the latest commit, interpret the Jenkinsfile, and run the deployment batch script.
- Opening C:/Deployments/DemoApp/index.html in a web browser verifies that the initial frontend file was deployed successfully.

Step 5: Continuous Integration & Continuous Deployment (CI/CD) Verification

- To simulate real-world continuous deployment, a code change is made directly in index.html on GitHub by adding custom CSS styling and modifying the text to Application Updated & Deployed via CI/CD Pipeline! Version 2.0 is live.
- Upon running Build #3, Jenkins pulls the updated commit hash, executes the stages, and overwrites the existing index.html in C:\Deployments\DemoApp with the new version.
- Refreshing the browser page confirms that the new blue-themed card UI and Version 2.0 text are rendered, validating the full automated delivery pipeline from source control to local target deployment.

Output

Step 1 — Jenkins server initialization and access setup



Fig. 1 — Verifying the Java installation (java --version)

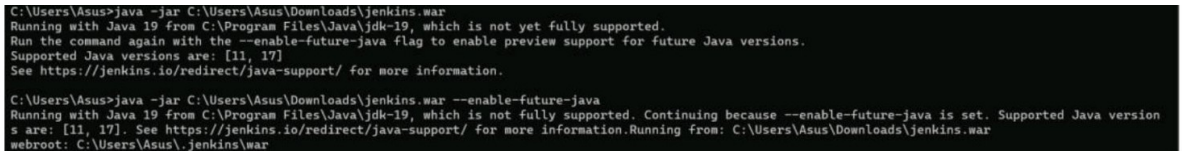


Fig. 2 — Starting Jenkins from the terminal using the WAR file

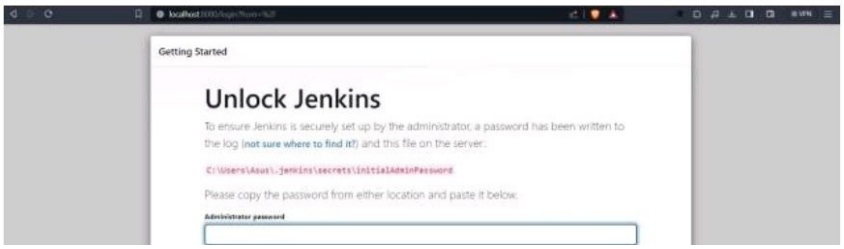


Fig. 3 — Opening http://localhost:8080 — Unlock Jenkins screen

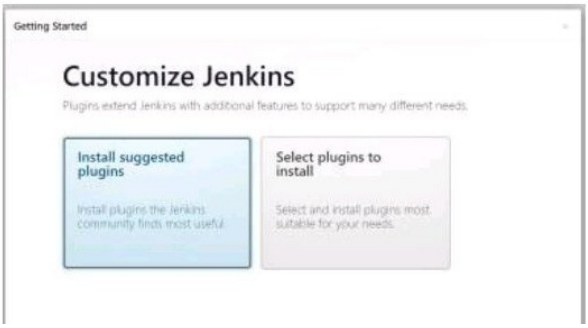


Fig. 4 — Customize Jenkins — selecting "Install suggested plugins"

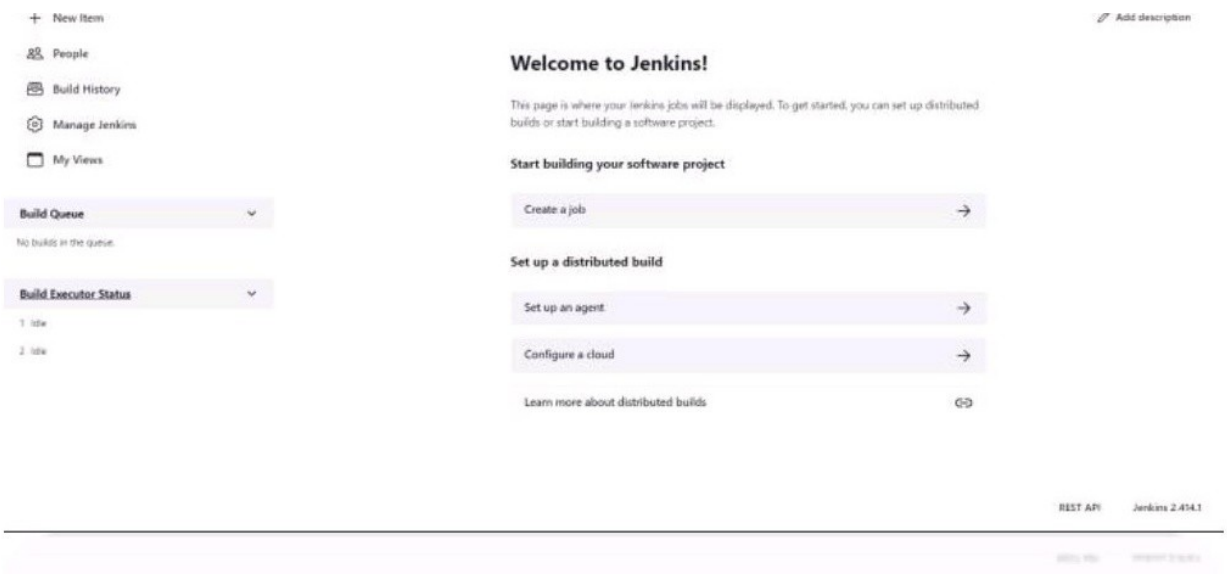


Fig. 5 — Jenkins dashboard after setup is complete

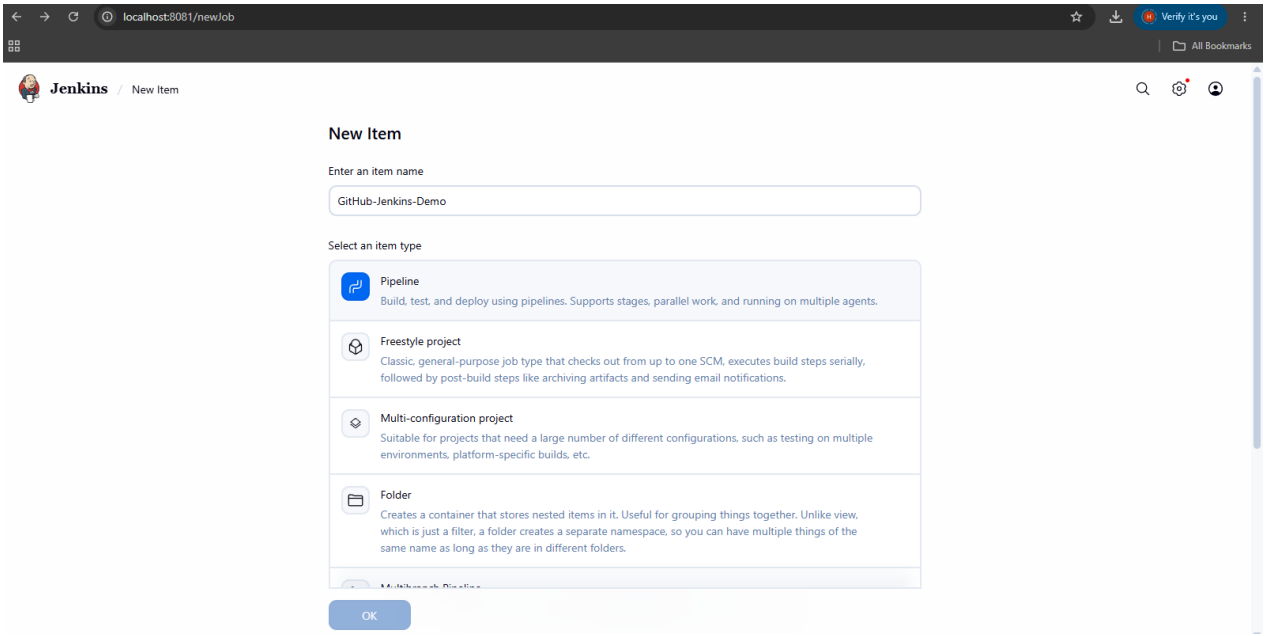


Fig. 6 — Step 2 — Creating the GitHub-Jenkins-Demo item (Pipeline type)

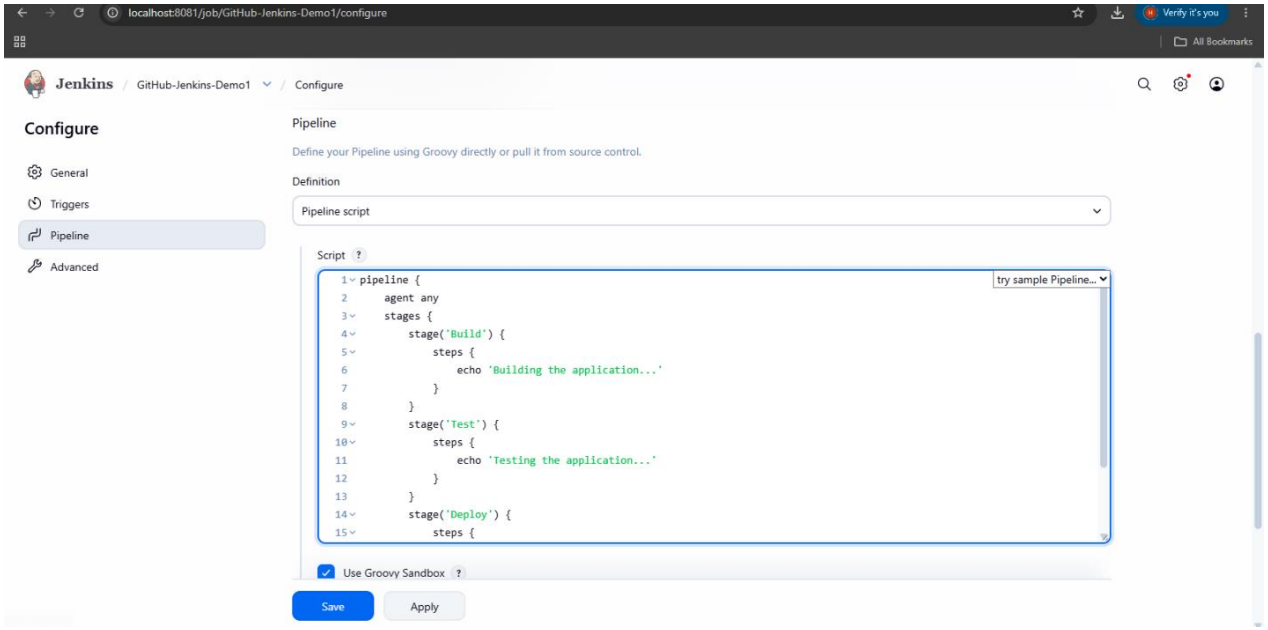


Fig. 7 — Step 2 — Inline Declarative Pipeline script (Build, Test, Deploy)

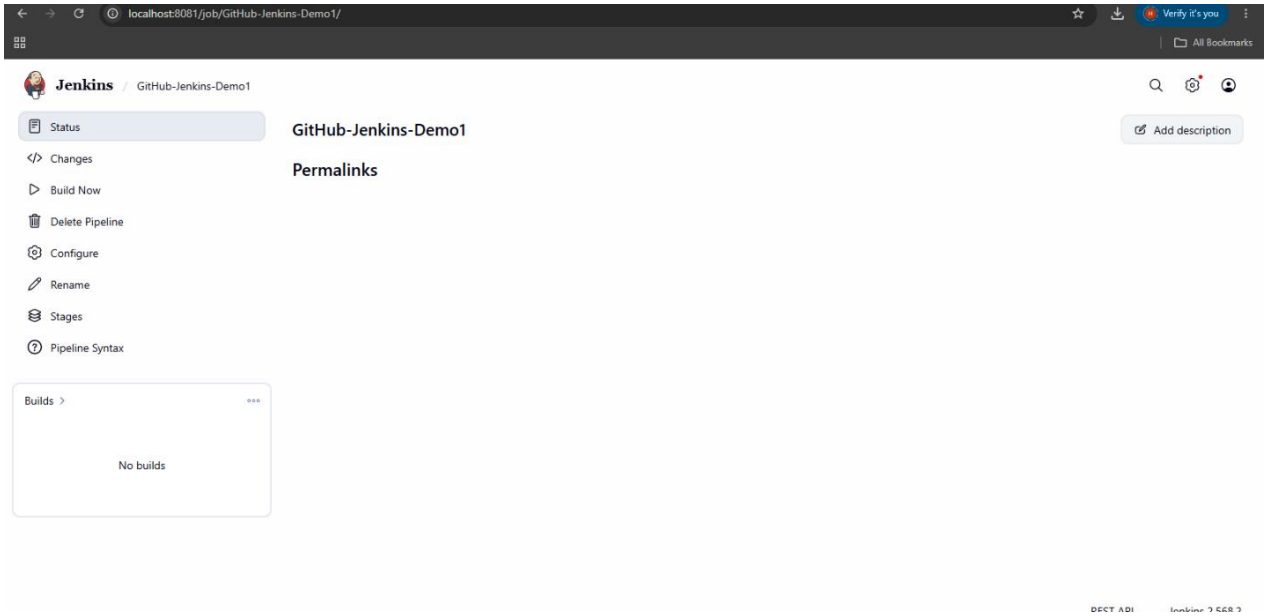


Fig. 8 — Step 2 — GitHub-Jenkins-Demo job status page

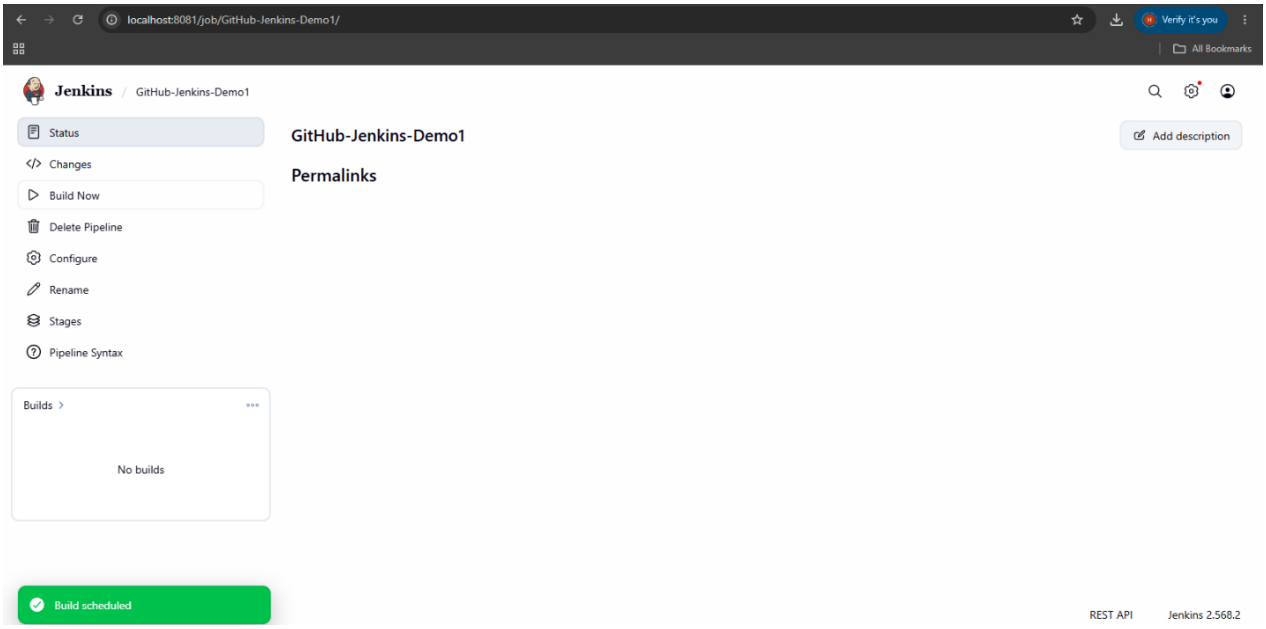


Fig. 9 — Step 2 — Build scheduled from the job page

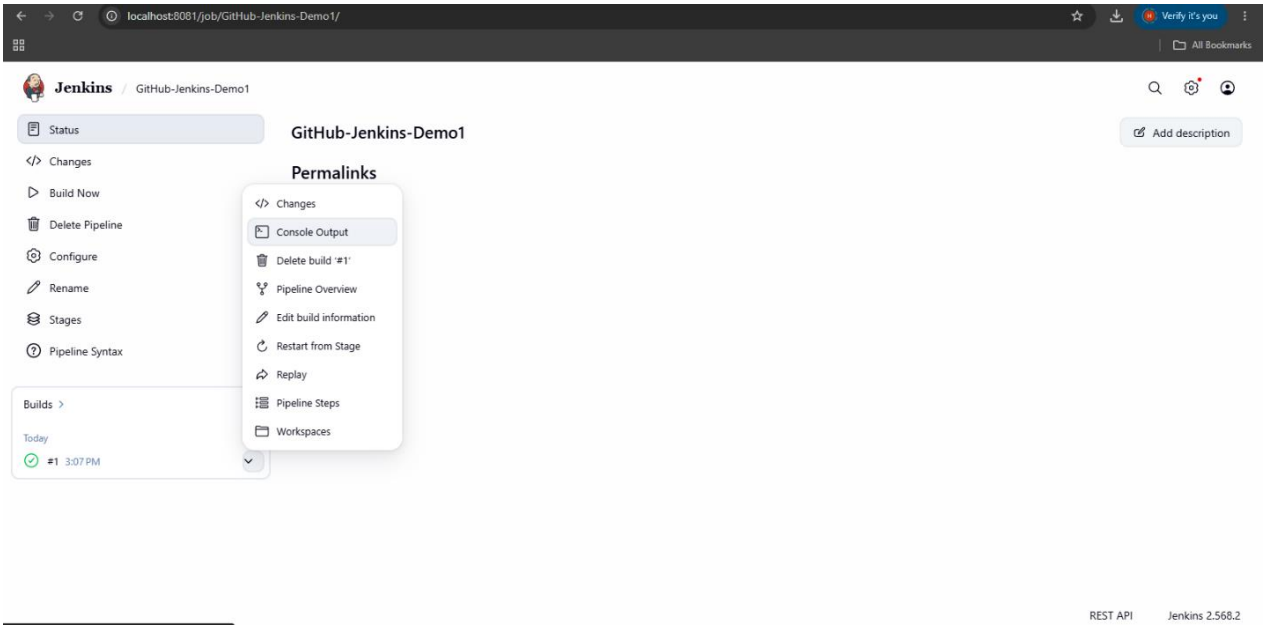


Fig. 10 — Step 2 — Build #1 completed; opening Console Output

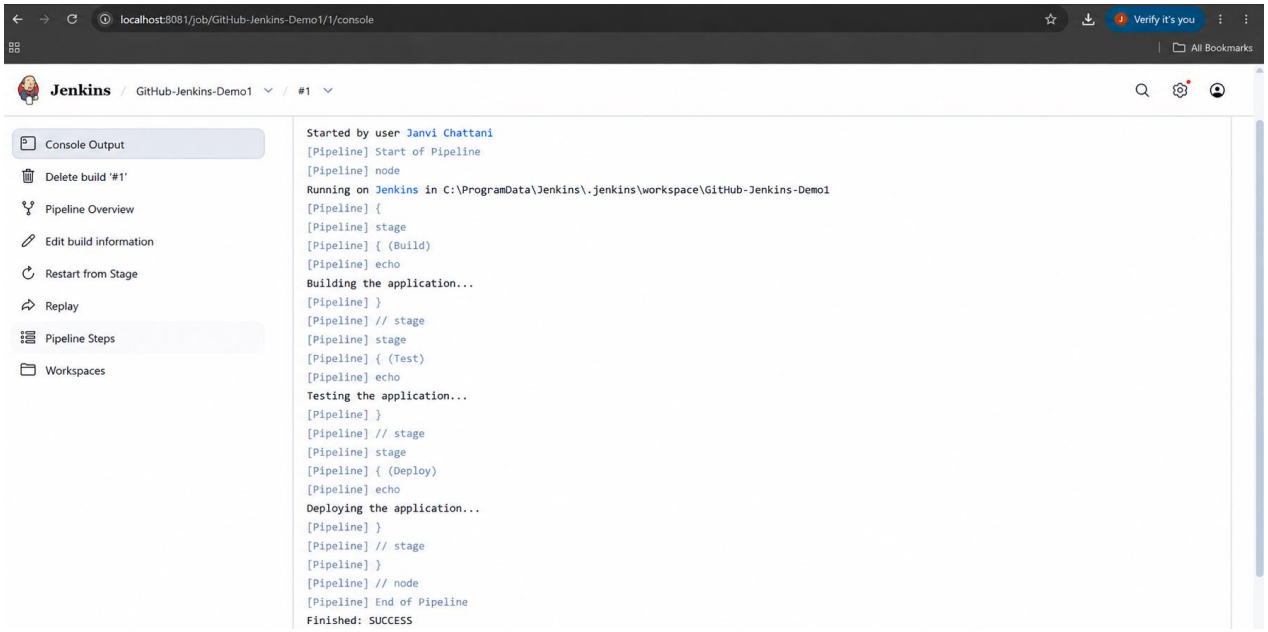


Fig. 11 — Step 2 — Build #1 console output ending in SUCCESS

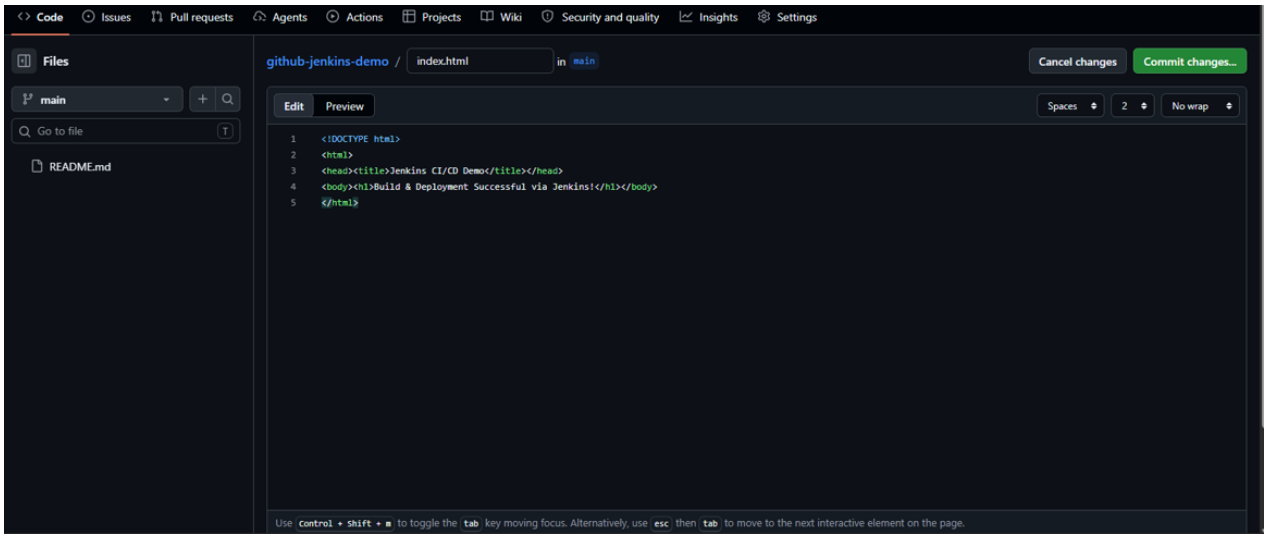


Fig. 12 — Step 3 — index.html created in the GitHub repository

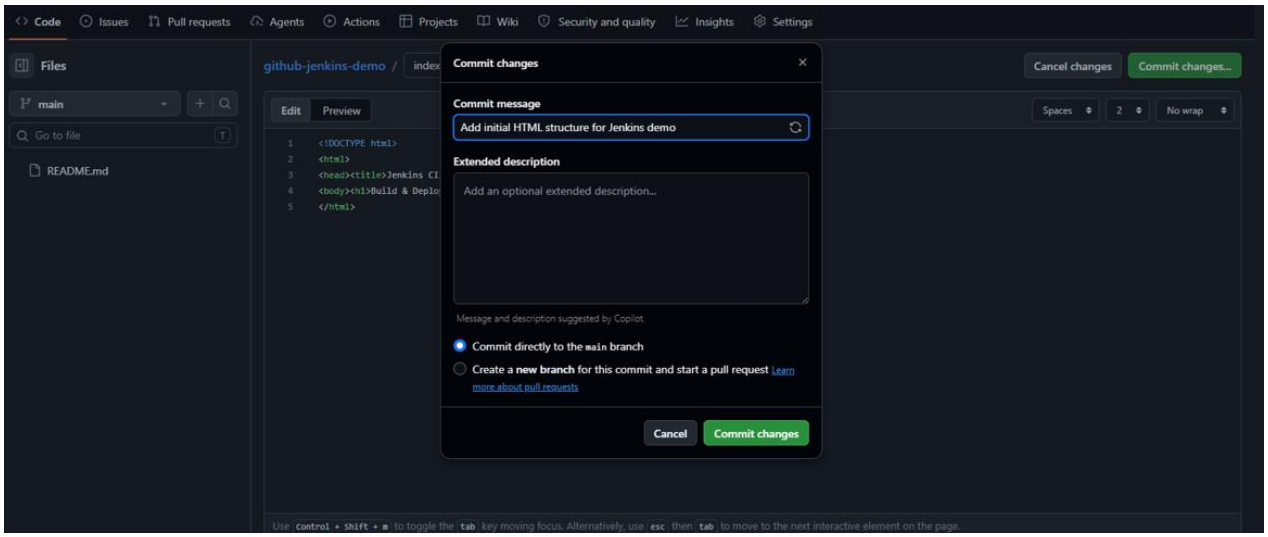


Fig. 13 — Step 3 — Committing index.html to the main branch

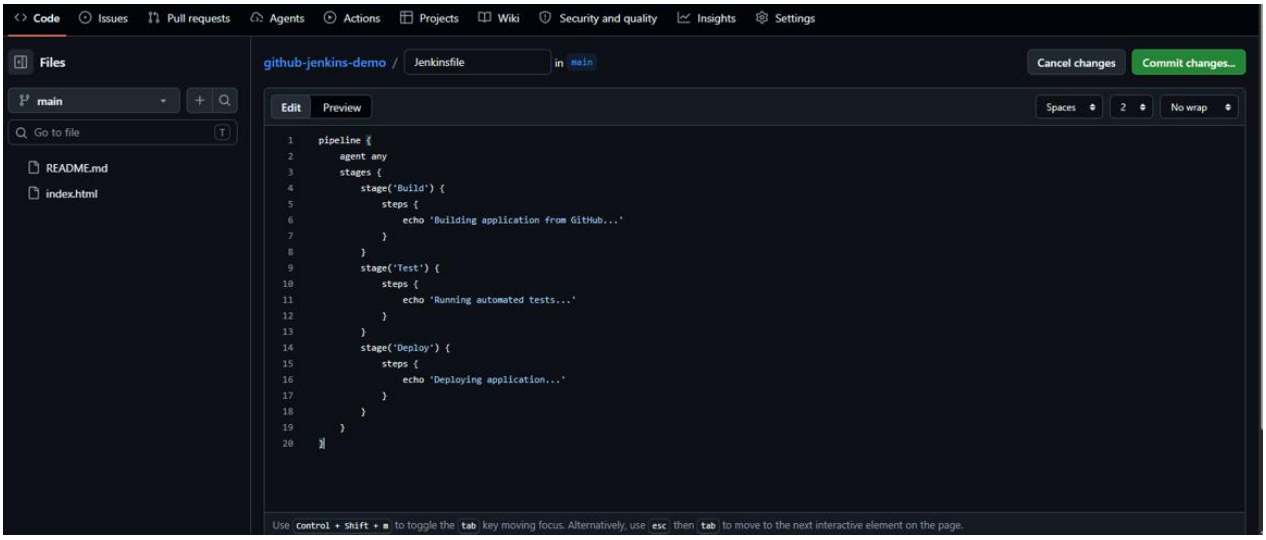


Fig. 14 — Step 3 — Jenkinsfile created in the repository root

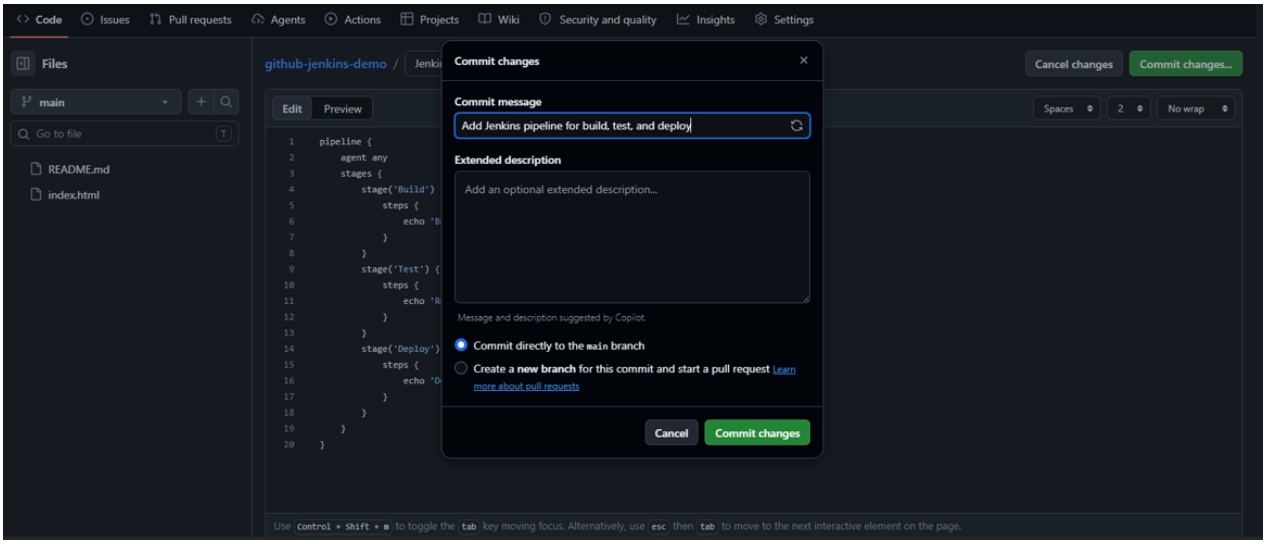


Fig. 15 — Step 3 — Committing the Jenkinsfile to the main branch



Fig. 16 — Step 3 — Jenkins job trigger and pipeline configuration

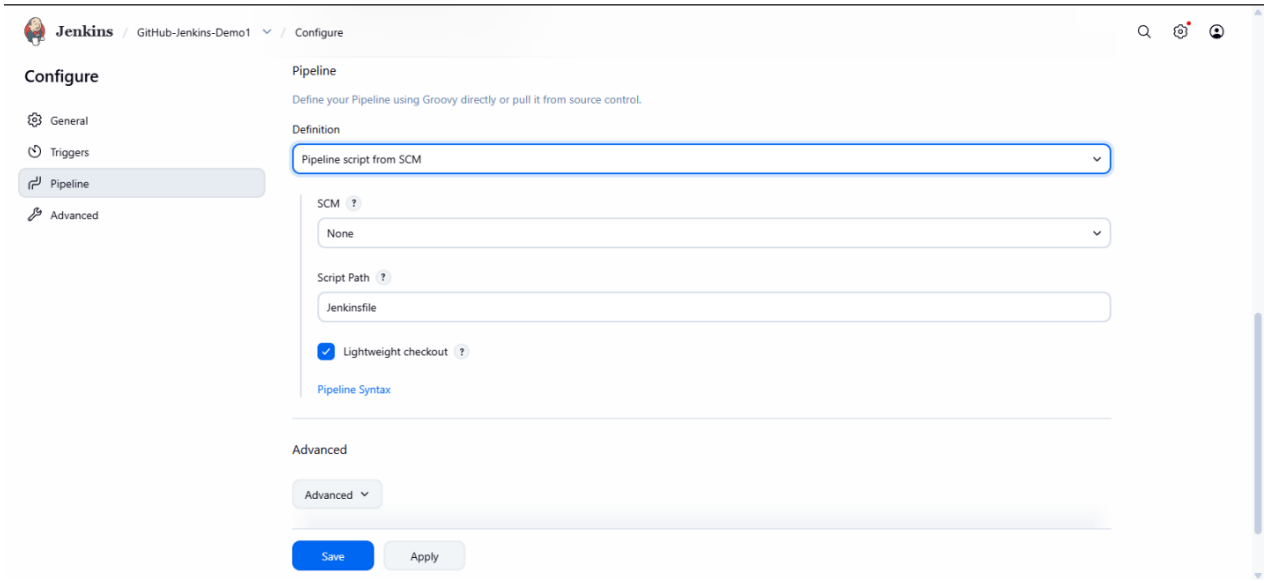


Fig. 17 — Step 4 — Definition changed to "Pipeline script from SCM"

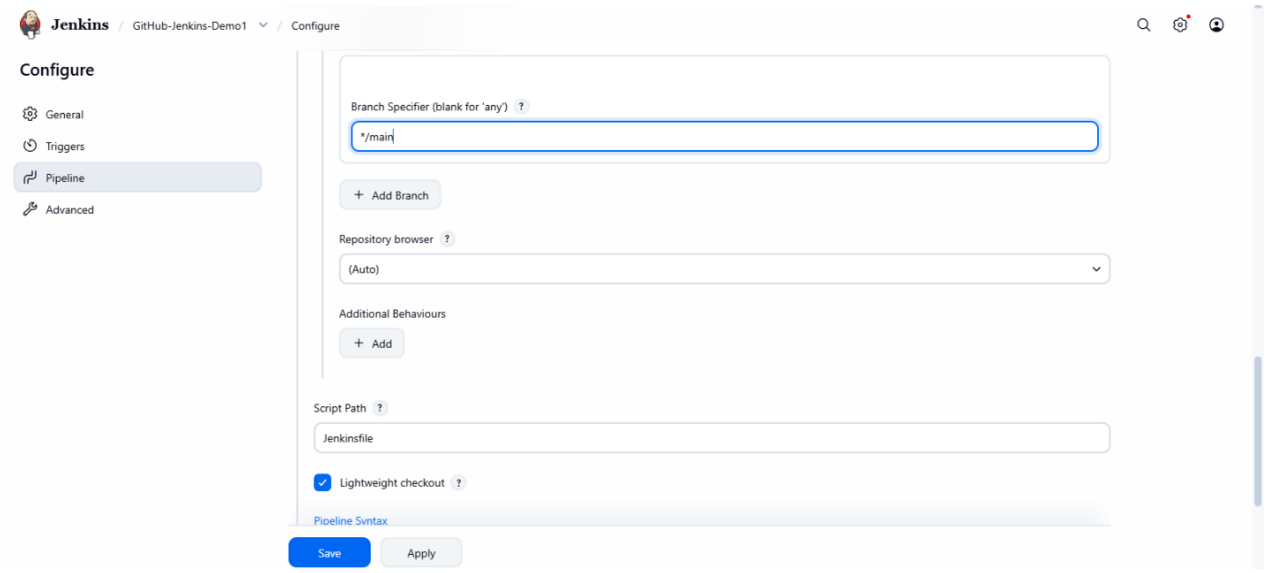


*Fig. 18 — Step 4 — Branch specifier set to */main, script path Jenkinsfile*

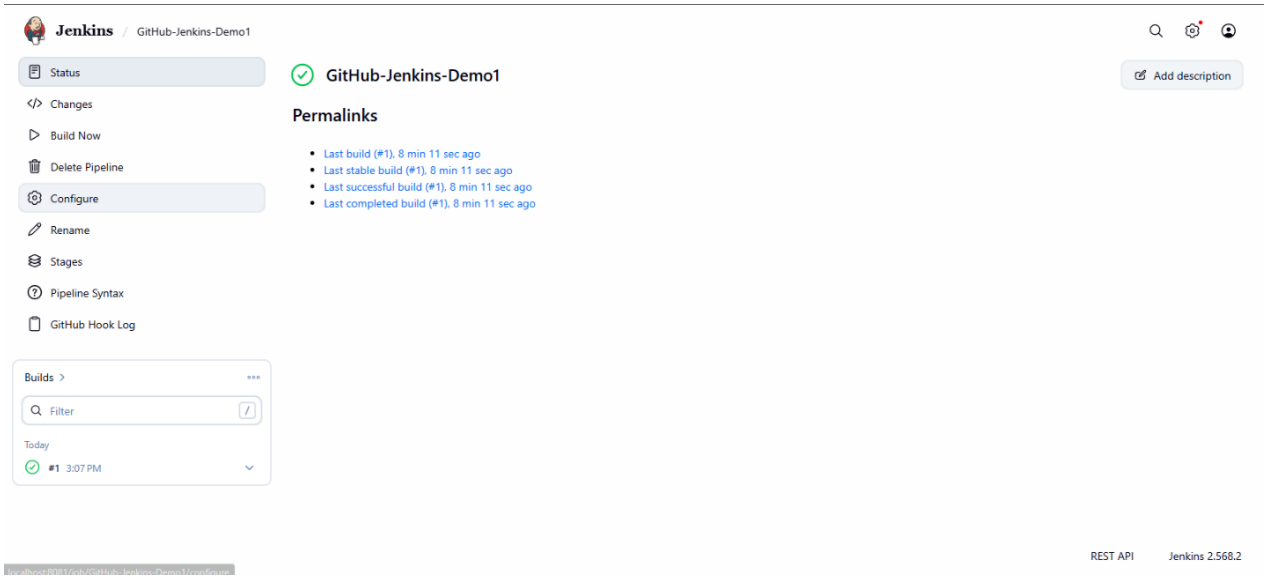


Fig. 19 — Step 4 — Job status after Build #1

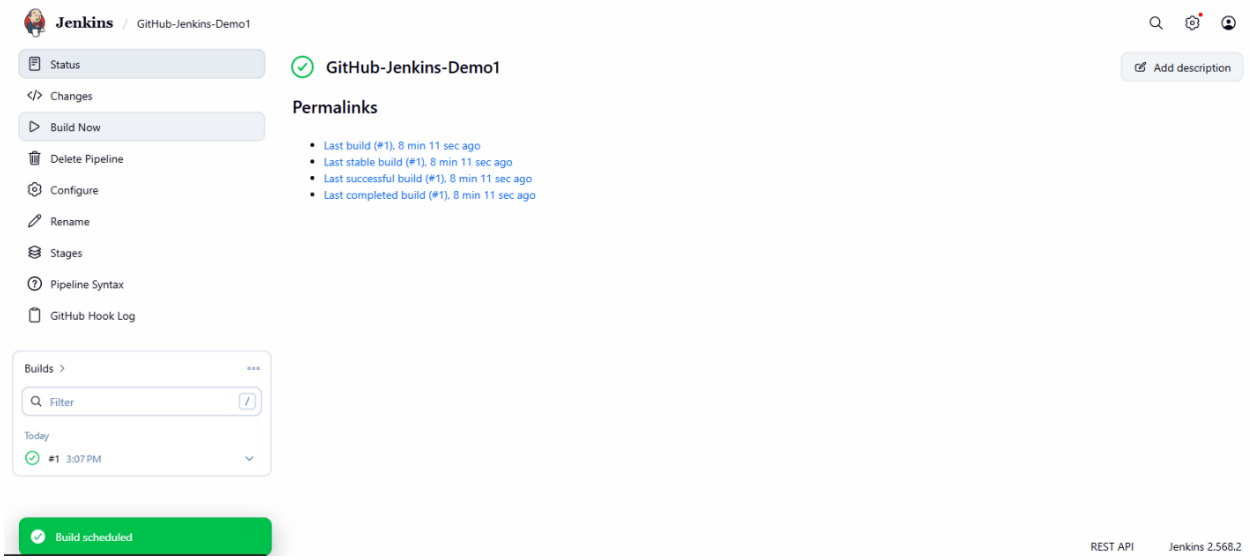


Fig. 20 — Step 4 — Build #2 scheduled

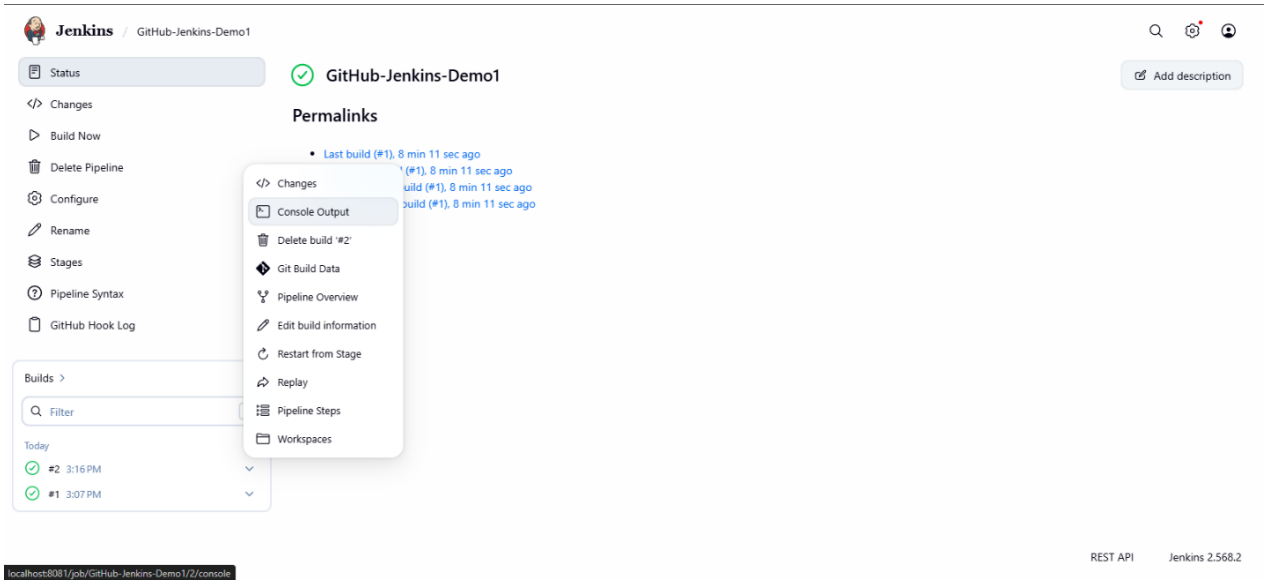


Fig. 21 — Step 4 — Opening Build #2 console output

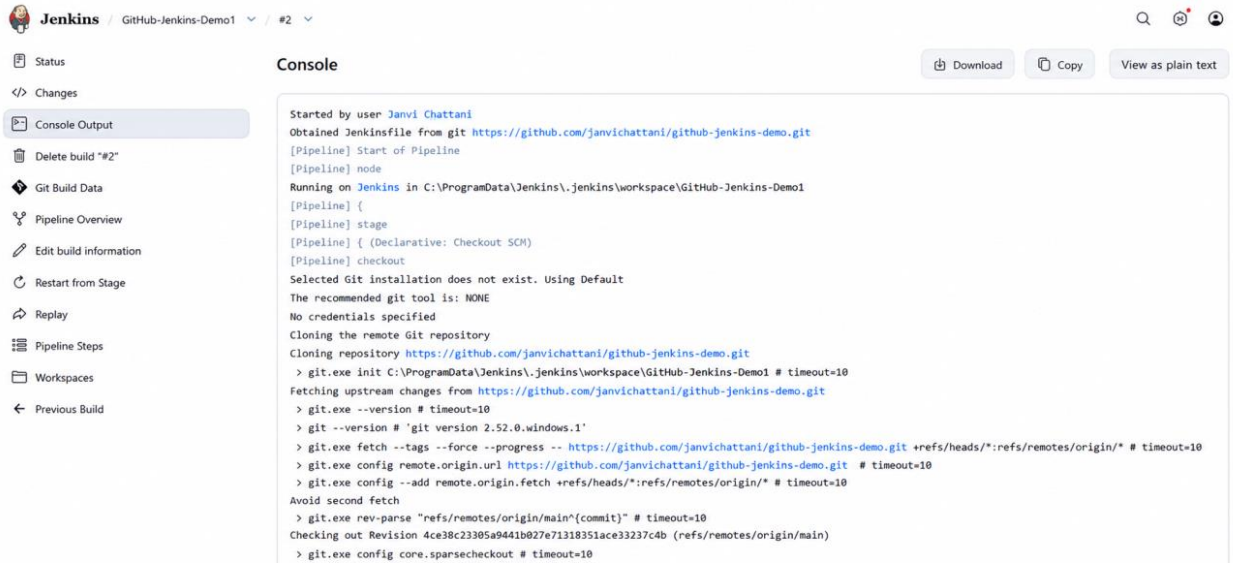


Fig. 22 — Step 4 — Build #2 console: cloning the repository and checking out the commit

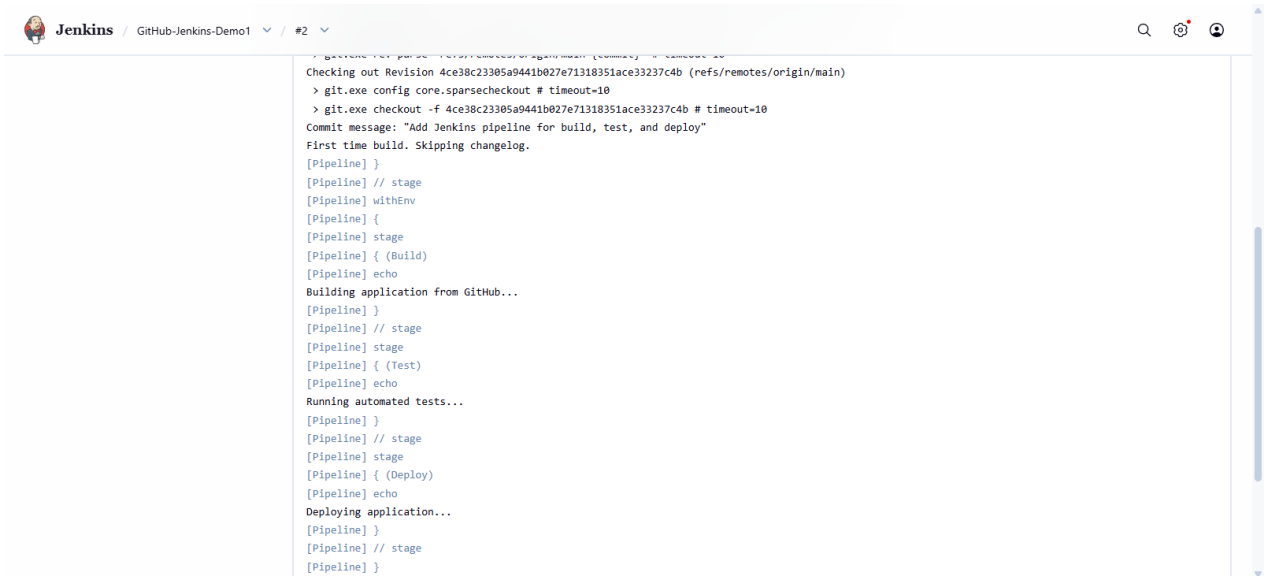


Fig. 23 — Step 4 — Build #2 console: Build, Test and Deploy stages



Fig. 24 — Step 4 — Build #2 console ending in SUCCESS

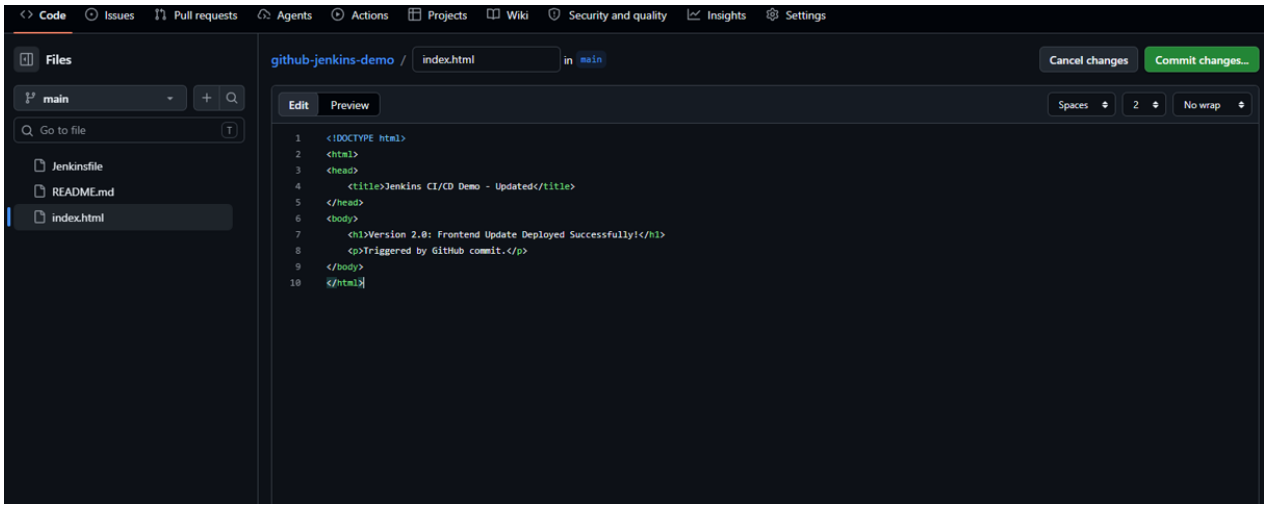


Fig. 25 — Step 5 — index.html edited on GitHub (Version 2.0)

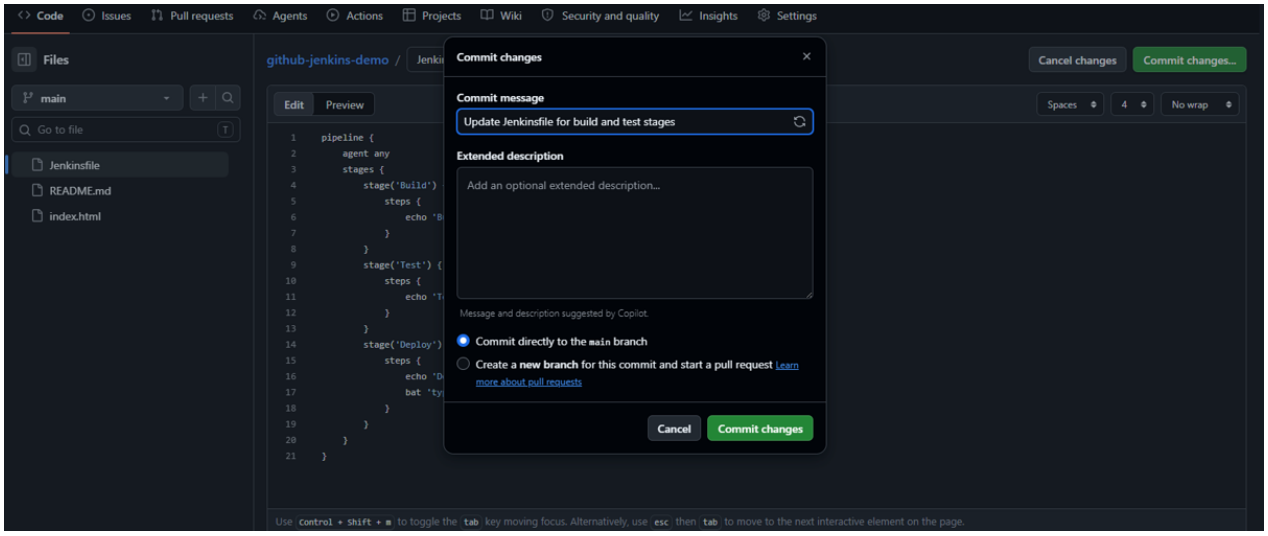


Fig. 26 — Step 5 — Committing the Jenkinsfile update

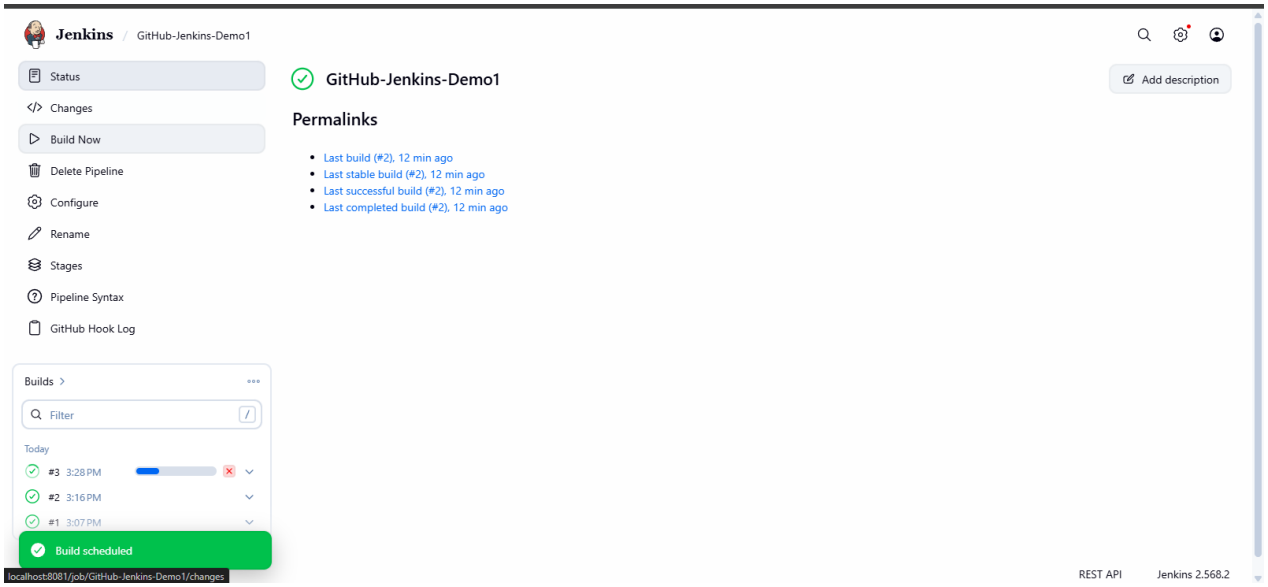


Fig. 27 — Step 5 — Build #3 triggered

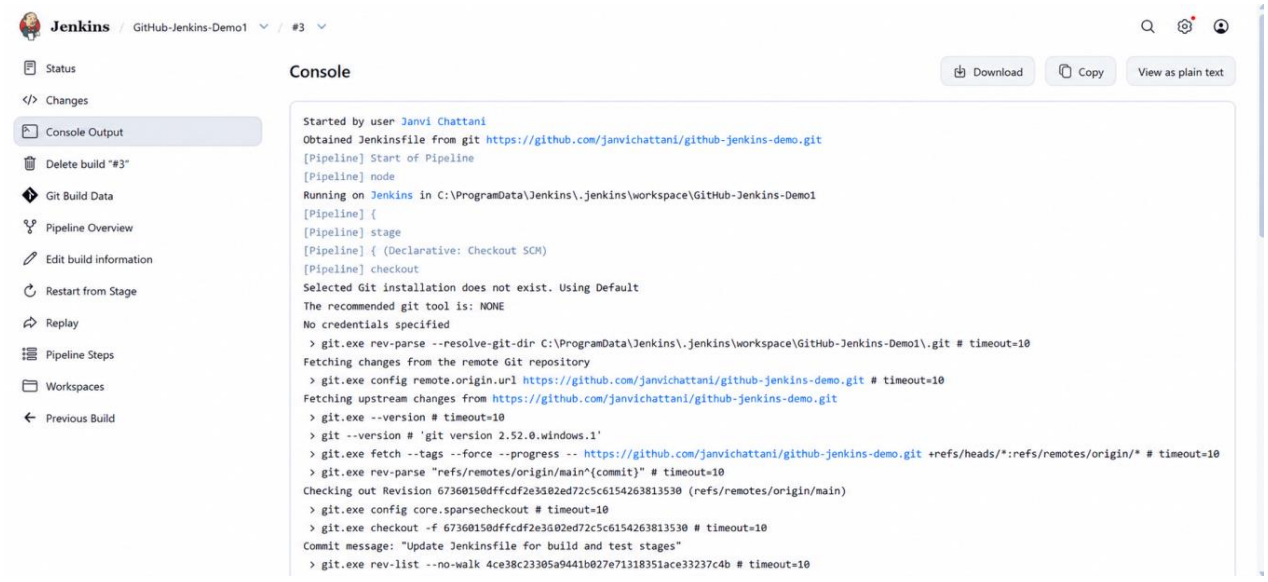


Fig. 28 — Step 5 — Build #3 console output after the new commit



Fig. 29 — Step 5 — Build #3 Deploy stage and SUCCESS



Fig. 30 — Step 5 — Browser showing the updated page (Version 2.0)